

网络流算法及其应用

5.1 基本概念

在实际生活中有许多流量问题，例如在交通运输网络中的人流、车流、货物流，供水网络中的水流，金融系统中的现金流，通讯系统中的信息流，等等。50年代以福特(Ford)、富克逊(Fulkerson)为代表建立的“网络流理论”，是网络应用的重要组成部分。在最近的奥林匹克信息学竞赛中，利用网络流算法高效地解决问题已不是什么稀罕的事了。本节着重介绍最大流(包括最小费用)算法，并通过实际例子，讨论如何在问题的原型上建立一个网络流模型，然后用最大流算法高效地解决问题。

1.问题描述 如图 5-1 所示是联结某产品地 v_1 和销售地 v_4 的交通网，每一弧 (v_i, v_j) 代表从 v_i 到 v_j 的运输线，产品经这条弧由 v_i 输送到 v_j ，弧旁的数表示这条运输线的最大通过能力。产品经过交通网从 v_1 到 v_4 。现在要求制定一个运输方案使从 v_1 到 v_4 的产品数量最多。

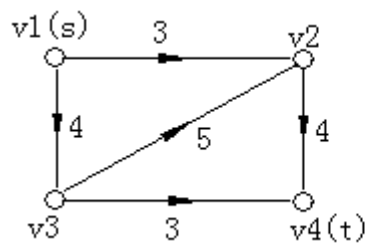


图 5-1

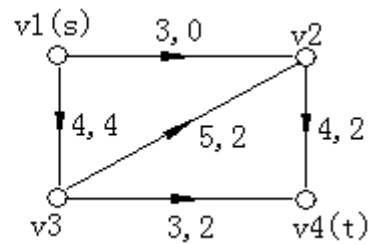


图 5-2

2.网络与网络流

给一个有向图 $N=(V, E)$ ，在 V 中指定一点，称为源点(记为 v_s ，和另一点，称为汇点(记为 v_t)，其余的点叫中间点，对于 E 中每条弧 (v_i, v_j) 都对应一个正整数 $c(v_i, v_j) \geq 0$ (或简写成 c_{ij})，称为 f 的容量，则赋权有向图 $N=(V, E, c, v_s, v_t)$ 称为一个网络。如图 5-1 所给出的一个赋权有向图 N 就是一个网络，指定 v_1 是源点， v_4 为汇点，弧旁的数字为 c_{ij} 。所谓网络上的流，是指定义在弧集合 E 上一个函数 $f=\{f(v_i, v_j)\}$ ，并称 $f(v_i, v_j)$ 为弧 (v_i, v_j) 上的流量(下面简记为 f_{ij})。如图 5-2 所示的网络 N ，弧上两个数，第一个数表示容量 c_{ij} ，第二个数表示流量 f_{ij} 。

3.可行流与最大流

在运输网络的实际问题中，我们可以看出，对于流有两个显然的要求：一是每个弧上的流量不能超过该弧的最大通过能力(即弧的容量)；二是中间点的流量为 0，源点的净流出量和汇点的净流入量必相等且为这个方案的总输送量。因此有：

(1)容量约束： $0 \leq f_{ij} \leq c_{ij}$ ， $(v_i, v_j) \in E$ ，

(2)守恒条件

对于中间点：流入量=流出量；对于源点与汇点：源点的净流出量 $v_s(f)$ =汇点的净流入量 $(-v_t(f))$ 的流 f ，称为网络 N 上的可行流，并将源点 s 的净流量称为流 f 的流值 $v(f)$ 。

网络 N 中流值最大的流 f^* 称为 N 的最大流。

4.可增广路径

所谓可增广路径，是指这条路径上的流可以修改，通过修改，使得整个网络的流值增大。

设 f 是一个可行流， P 是从源点 s 到汇点 t 的一条路，若 p 满足下列条件：

(1)在 p 上的所有前向弧 $(v_i \rightarrow v_j)$ 都是非饱和弧，即 $0 \leq f_{ij} < c_{ij}$

(2)在 p 上的所有后向弧 $(v_i \leftarrow v_j)$ 都是非零弧，即 $0 < f_{ij} \leq c_{ij}$

则称 p 为(关于可行流 f 的)一条可增广路径。

5.最大流定理

当且仅当不存在关于 f^* 的增广路径，可行流 f^* 为最大流。

5.2 最大流算法

算法思想:最大流问题实际上是求一可行流 $\{f_{ij}\}$ ，使得 $v(f)$ 达到最大。若给了一个可行流 f ，只要判断 N 中有无关于 f 的增广路径，如果有增广路径，改进 f ，得到一个流量增大的新的可行流；如果没有增广路径，则得到最大流。

1.寻求最大流的标号法(Ford,Fulkerson)

从一个可行流(一般取零流)开始，不断进行以下的标号过程与调整过程，直到找不到关于 f 的可增广路径为止。

(1)标号过程

在这个过程中，网络中的点分为已标号点和未标号点，已标号点又分为已检查和未检查两种。每个标号点的标号信息表示两个部分：第一标号表明它的标号从哪一点得到的，以便从 v_t 开始反向追踪找出也增广路径；第二标号是为了表示该顶点是否已检查过。

标号开始时，给 v_s 标上 $(s, 0)$ ，这时 v_s 是标号但未检查的点，其余都是未标号的点，记为 $(0, 0)$ 。

取一个标号而未检查的点 v_i ，对于一切未标号的点 v_j ：

A. 对于弧 (v_i, v_j) ，若 $f_{ij} < c_{ij}$ ，则给 v_j 标号 $(v_i, 0)$ ，这时， v_j 点成为标号而未检查的点。

B. 对于弧 (v_i, v_j) ，若 $f_{ji} > 0$ ，则给 v_j 标号 $(-v_i, 0)$ ，这时， v_j 点成为标号而未检查的点。

于是 v_i 成为标号且已检查的点，将它的第二个标号记为 1。重复上述步骤，一旦 v_t 被标上号，表明得到一条从 v_i 到 v_t 的增广路径 p ，转入调整过程。

若所有标号都已检查过去，而标号过程进行不下去时，则算法结束，这时的可行流就是最大流。

(2)调整过程

从 v_t 点开始，通过每个点的第一个标号，反向追踪，可找出增广路径 P 。例如设 v_t 的第一标号为 v_k (或 $-v_k$)，则弧 (v_k, v_t) (或相应地 (v_t, v_k)) 是 p 上弧。接下来检查 v_k 的第一标号，若为 v_i (或 $-v_i$)，则找到 (v_i, v_k) (或相应地 (v_k, v_i))。再检查 v_i 的第一标号，依此类推，直到 v_s 为止。这时整个增广路径就找到了。在上述找增广路径的同时计算 Q ：

$$Q = \min \{ \min(c_{ij} - f_{ij}), \min f^*_{ij} \}$$

对流 f 进行如下的修改：

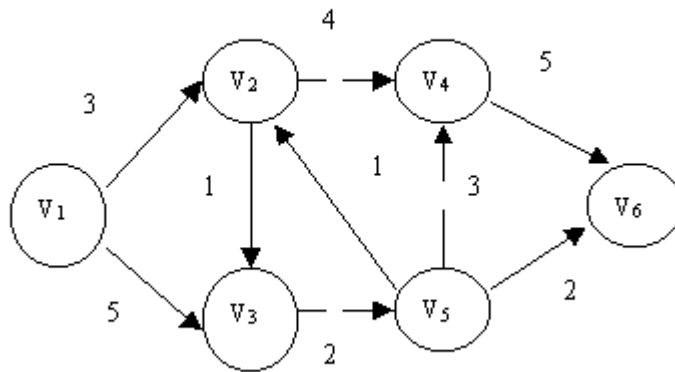
$$f_{ij} = f_{ij} + Q \quad (v_i, v_j) \in P \text{ 的前向弧的集合}$$

$$f_{ij} = f_{ij} - Q \quad (v_i, v_j) \in P \text{ 的后向弧的集合}$$

$$f_{ij} = f^*_{ij} \quad (v_i, v_j) \text{ 不属于 } P \text{ 的集合}$$

接着，清除所有标号，对新的可行流 f' ，重新进入标号过程。

例 1:下图表示一个公路网, V_1 是某原材料产地, V_6 表示港口码头, 每段路的通过能力(容量)如图上的各边上的数据, 找一运输方案, 使运输到码头的原材料最多?



程序如下:

```

Program Max_Stream;
Const Maxn=20;
type
  nettype=record
    C,F:integer;
  end;
  nodetype=record
    L,P:integer;
  end;
var
  Lt:array[0..maxn] of nodetype;
  G:Array[0..Maxn,0..Maxn] of Nettype;
  N,S,T:integer;
  F:Text;
Procedure Init;{初始化过程，读入有向图，并设置流为 0}
Var Fn :String;
    I,J :Integer;
Begin
  Write( 'Graph File = ' ); Readln(Fn);
  Assign(F,Fn);
  Reset(F);
  Readln(F,N);
  Fillchar(G,Sizeof(G),0);
  Fillchar(Lt,Sizeof(Lt),0);
  For I:=1 To N Do
    For J:=1 To N Do Read(F,G[I,J].C);
  Close(F);
End;
Function Find: Integer; {寻找已经标号未检查的顶点}
Var I: Integer;

```

```

Begin
  I:=1;
  While (I<=N) And Not((Lt[I].L<>0)And(Lt[I].P=0)) Do Inc(I);
  If I>N Then Find:= 0 Else Find:= I;
End;
Function Ford(Var A: Integer):Boolean;
Var {用标号法找增广路径，并求修改量 A}
  I,J,M,X:Integer;
Begin
  Ford:=True;
  Fillchar(Lt,Sizeof(Lt),0);
  Lt[S].L:=S;
  Repeat
    I:= Find;
    If i=0 Then Exit;
    For J:=1 To N Do
      If (Lt[J].L= 0)And((G[I,J].C<>0)or(G[J,I].C<>0)) Then
        Begin
          if (G[I,J].F<G[I,J].C) Then Lt[J].L:= I;
          If (G[J,I].F>0) Then Lt[J].L:=-I;
        End;
        Lt[I].P:=1;
      Until (Lt[T].L<>0);
      M:=T;A:=Maxint;
      Repeat
        J:=M;M:=Abs(Lt[J].L);
        If Lt[J].L<0 Then X:= G[J,M].F;
        If Lt[J].L>0 Then X:= G[M,J].C- G[M,J].F;
        If X<A Then A:= X;
      Until M= S;
      Ford:=False;
    End;
  Procedure Change(A: Integer);{调整过程}
  Var M, J: Integer;
  Begin
    M:= T;
    Repeat
      J:=M;M:=Abs(Lt[J].L);
      If Lt[J].L<0 Then G[J,M].F:=G[J,M].F-A;
      If Lt[J].L>0 Then G[M,J].F:=G[M,j].F+A;
    Until M=S;
  End;
  Procedure Print; {打印最大流及其方案}
  VAR

```

```

    I,J: Integer;
    Max: integer;
Begin
    Max:=0;
    For I:=1 To N DO
        Begin
            If G[I,T].F<>0 Then Max:= Max + G[I,T].F;
            For J:= 1 To N Do
                If G[I,J].F<>0 Then Writeln( I,'->',J,' ',G[I,J].F);
            End;
            Writeln('The Max Stream=',Max);
        End;
    Procedure Process;{求最大流的主过程}
    Var Del:Integer;
        Success:Boolean;
    Begin
        S:=1;T:=N;
        Repeat
            Success:=Ford(Del);
            If Success Then Print
            Else Change(Del);
        Until Success;
    End;
    Begin {Main Program}
        Init;
        Process;
    End.

```

测试数据文件(zdl.txt)如下:

```

6
0 3 5 0 0 0
0 0 1 4 0 0
0 0 0 0 2 0
0 0 0 0 0 5
0 1 0 3 0 2
0 0 0 0 0 0

```

运行结果如下:

Graph File=zdl.txt

```

1->2  3
1->3  2
2->4  3
3->5  2
4->6  3
5->6  2

```

The Max Stream=5

5.3 最小费用最大流及算法

上面的网络，可看作为辅送一般货物的运输网络，此时，最大流问题仅表明运输网络运输货物的能力，但没有考虑运送货物的费用。在实际问题中，运送同样数量货物的运输方案可能有多个，因此从中找一个输出费用最小的方案是一个很重要的问题，这就是最小代价流所要讨论的内容。

1. 最小费用最大流问题的模型

给定网络 $N=(V, E, c, w, s, t)$ ，每一弧 (v_i, v_j) 属于 E 上，除了已给容量 c_{ij} 外，还给了一个单位流量的费用 $w(v_i, v_j) \geq 0$ (简记为 w_{ij})。所谓最小费用最大流问题就是要求一个最大流 f ，使得流的总输送费用取最小值。

$$W(f) = \sum w_{ij} f_{ij}$$

2. 用对偶法解最小费用最大流问题

定义：已知网络 $N=(V, E, c, w, s, t)$ ， f 是 N 上的一个可行流， p 为 v_s 到 v_t (关于流 f) 的可增广路径，称 $W(p) = \sum w_{ij}(p^+) - \sum w_{ij}(p^-)$ 为路径 p 的费用。

若 p^* 是从 v_s 到 v_t 所有可增广路径中费用最小的路径，则称 p^* 为最小费用可增广路径。

定理：若 f 是流量为 $v(f)$ 的最小费用流， p 是关于 f 的从 v_s 到 v_t 的一条最小费用可增广路径，则 f 经过 p 调整流量 Q 得到新的可行流 f' (记为 $f' = f + Q$)，一定是流量为 $v(f) + Q$ 的可行流中的最小费用流。

3. 对偶法的基本思路

①取 $f = \{0\}$

②寻找从 v_s 到 v_t 的一条最小费用可增广路径 p 。

若不存在 p ，则 f 为 N 中的最小费用最大流，算法结束。

若存在 p ，则用求最大流的方法将 f 调整成 f^* ，使 $v(f^*) = v(f) + Q$ ，并将 f^* 赋值给 f ，转②。

4. 迭代法求最小费用可增广路径

在前一节中，我们已经知道了最大流的求法。在最小费用最大流的求解中，每次要找一条最小费用的增广路径，这也是与最大流求法唯一不同之处。于是，对于求最小费用最大流问题余下的问题是怎样寻求关于 f 的最小费用增广路径 p 。为此，我们构造一个赋权有向图 $b(f)$ ，它的顶点是原网络 N 中的顶点，而把 N 中每一条弧 (v_i, v_j) 变成两个相反方向的弧 (v_i, v_j) 和 (v_j, v_i) 。定义 $w(f)$ 中的弧的权如下：

如果 $(f_{ij} < c_{ij})$ ，则 $b_{ij} = w_{ij}$ ；如果 $(f_{ij} = c_{ij})$ ，则 $b_{ij} = +\infty$

如果 $(f_{ij} > 0)$ ，则 $b_{ji} = -w_{ij}$ ；如果 $(f_{ij} = c_{ij})$ ，则 $b_{ji} = +\infty$

于是在网络 N 中找关于 f 的最小费用增广路径就等价于在赋权有向图 $b(f)$ 中，寻求从 v_s 到 v_t 的最短路径。求最短路径有三种经典算法，它们分别是 Dijkstra 算法、Floyd 算法和迭代算法 (Ford 算法)。由于在本问题中，赋权有向图 $b(f)$ 中存在负权，故我们只能用后两种方法求最短路径，其中对于本问题最高效的算法是迭代算法。为了程序的实现方便，我们只要对原网络做适当的调整。将原网络中的每条弧 (v_i, v_j) 变成两条相反的弧：

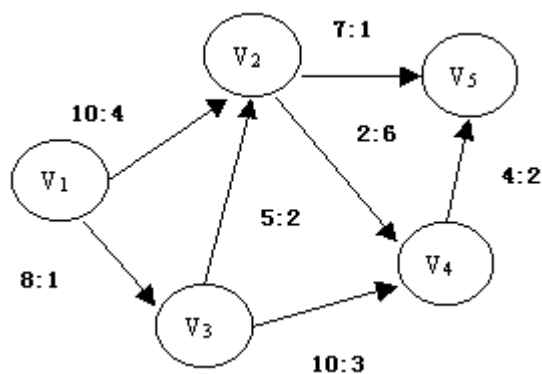
前向弧 (v_i, v_j) ，其容量 c_{ij} 和费用 w_{ij} 不变，流量为 f_{ij} ；

后向弧 (v_j, v_i) ，其容量 0 和费用 $-w_{ij}$ ，流量为 $-f_{ij}$ 。

事实上，对于原网络的数据结构中，这些单元已存在，在用标号法求最大流时是闲置的。这样我们就不必产生关于流 f 的有向图 $b(f)$ 。同时对判断 (v_i, v_j) 的流量可否改时，可以统一为判断是否 “ $f_{ij} < c_{ij}$ ”。因为对于后向弧 (v_j, v_i) ，若 $f_{ij} > 0$ ，则 $f_{ji} = -f_{ij} < 0 = c_{ji}$ 。

例 2：求输送量最大且费用最小的运输方案？

如下图是一公路网， v_1 是仓库所在地（物资的起点）， v_5 是某一工地（物资的终点）每条弧旁的两个数字分别表示某一时间内通过该段路的最多吨数和每吨物资通过该段路的费用。



程序如下：

{最小费用最大流参考程序}

```
program Maxflow_With_MinCost;
```

```
const
```

```
    name1='flow.in';
```

```
    name2='flow.out';
```

```
    maxN=100;{最多顶点数}
```

```
type
```

```
    Tbest=record    {数组的结构}
```

```
        value,fa:integer;
```

```
    end;{到源点的最短距离，父辈}
```

```
    Nettype=record
```

```
        {网络邻接矩阵类型}
```

```
        c,w,f:integer;
```

```
        {弧的容量，单位通过量费用，流量}
```

```
    end;
```

```
var
```

```
    Net:array[1..maxN,1..maxN] Of Nettype;
```

```
        {网络 N 的邻接矩阵}
```

```
    n,s,t:integer;{顶点数，源点、汇点的编号}
```

```
    Minw:integer;{最小总费用}
```

```
    best:array[1..maxN] of Tbest;{求最短路时用的数组}
```

```
procedure Init;{数据读入}
```

```
var inf:text;
```

```
    a,b,c,d:integer;
```

```
begin
```

```
    fillchar(Net,sizeof(Net),0);
```

```
    Minw:=0;
```

```
    assign(inf,name1);
```

```
    reset(inf);
```

```
    readln(inf,n);
```

```
    s:=1;t:=n;{源点为 1，汇点 n}
```

```
    repeat
```

```
        readln(inf,a,b,c,d);
```

```

    if a+b+c+d>0 then
    begin
    Net[a,b].c:=c;{给弧(a,b)赋容量 c}
    Net[b,a].c:=0;{给相反弧(b,a)赋容量 0}
    Net[a,b].w:=d;{给弧(a,b)赋费用 d}
    Net[b,a].w:=-d;{给相反弧(b, a)赋费用-d}
    end;
until a+b+c+d=0;
close(inf);
end;
function Find_Path:boolean;
{求最小费用增广路函数,若 best[t].value<>MaxInt,
则找到增广路, 函数返回 true, 否则返回 false}
var Quit:Boolean;
    i,j:Integer;
begin
    for i:=1 to n do best[i].value:=Maxint;best[s].value:=0;
    {寻求最小费用增广路径前, 给 best 数组赋初值}
    repeat
        Quit:=True;
        for i:=1 to n do
            if best[i].Value < Maxint then
                for j:=1 to n do
                    if (Net[i,j].f < Net[i,j].c) and
                    (best[i].value + Net[i,j].w <
                    best[j].value) then
                        begin
                            best[j].value:=best[i].value + Net[i,j].w;
                            best[j].fa := i;
                            Quit:= False;
                        end;
                until Quit;
            if best[t].value<Maxint then find_path:=true else find_path:=false;
        end;
    procedure Add_Path;
    var i,j: integer;
    begin
        i:= t;
        while i <> s do
            begin
                j := best[i].fa;
                inc(Net[j,i].f); {增广路是弧的数量修改, 增量 1}
                Net[i,j].f:=-Net[j,i].f;{给对应相反弧的流量赋值-f}
                i:=j;
            end;
        end;
    end;
end;

```



```

        end;
        inc(Minw,best[t].value); {修改最小总费用的值}
    end;
    procedure Out; {输出最小费用最大流的费用及方案}
    var ouf: text;
        i,j: integer;
    begin
        assign(ouf,name2);
        rewrite(ouf);
        writeln(ouf,'Min w = ',Minw);
        for i := 1 to n do
            for j:= 1 to n do
                if Net[i,j].f > 0 then
                    writeln(ouf,i,'->',j,': ',
                        Net[i,j].w,'*',Net[i,j].f);
            close(ouf);
        end;
    Begin {主程序}
        Init;
        while Find_Path do Add_Path;
        {当找到最小费用增广路, 修改流}
        Out;
    end.

```

flow.in 如下:

```

5
1 2 10 4
1 3 8 1
2 4 2 6
2 5 7 1
3 2 5 2
3 4 10 3
4 5 4 2
0 0 0 0

```

运行结果 flow.out 如下:

```

Min w = 55
1 -> 2: 4*3
1 -> 3: 1*8
2 -> 5: 1*7
3 -> 2: 2*4
3 -> 4: 3*4
4 -> 5: 2*4

```

练习:

- 1.熟记上述两种算法。
- 2.将例 2 程序中的寻找费用最小增广路径的算法改成 Floyd 算法。

